⚙ SETUP DE ENVIRONMENT EN WINDOWS

Smart Hour Meter v3 - Desarrollo Local

Versión: 3.0
Plataforma: Windows 10/11
Python: 3.9+

📋 REQUISITOS PREVIOS

Software necesario

- ✅ Python 3.9 o superior (https://www.python.org/downloads/)
- ✅ Git (opcional, para clonar repositorio)
- ✅ SQL Server Management Studio (para testing de BD)
- ✅ ODBC Driver 17 for SQL Server (https://www.microsoft.com/en-us/download/details.aspx?id=56567)

Hardware (opcional para testing)

- Opcional: Raspberry Pi Zero W con LCD y botones (para testear en hardware real)

🚀 INSTALACIÓN PASO A PASO

PASO 1: Instalar Python (si no lo tienes)

```bash
# Descargar desde: https://www.python.org/downloads/
# Instalar con "Add Python to PATH" MARCADO

# Verificar instalación
python --version
pip --version
```

PASO 2: Descargar archivos del proyecto

Opción A: Si tienes Git

```bash
git clone https://github.com/tuusuario/smart-hour-meter.git
cd smart-hour-meter
```

Opción B: Descargar como ZIP manualmente

```
Descargar archivos en carpeta: C:\smart-hour-meter\
```

PASO 3: Crear Virtual Environment

En Windows (PowerShell o CMD):

```bash
# Ir a carpeta del proyecto
cd C:\smart-hour-meter

# Crear virtual environment
python -m venv venv

# Activar environment (PowerShell)
.\venv\Scripts\Activate.ps1

# O si usas CMD:
.\venv\Scripts\activate.bat

# Deberías ver (venv) al inicio de la línea
```

PASO 4: Instalar ODBC Driver 17

Crítico para conexión a SQL Server

```bash
# Descargar e instalar desde:
# https://www.microsoft.com/en-us/download/details.aspx?id=56567

# O usar Chocolatey (si lo tienes):
choco install odbc-driver-17-for-sql-server
```

PASO 5: Instalar dependencias Python

Con el virtual environment ACTIVADO:

```bash
# Instalar requirements
pip install -r requirements.txt

# Esperar a que termine (~2-3 minutos)

# Verificar instalación
pip list

# Deberías ver:
# pyodbc
# python-dotenv
# dataclasses-json
# (+ otras librerías)
```

PASO 6: Configurar archivo .env

```bash
# Copiar ejemplo
copy .env.example .env

# Editar con tu editor favorito
notepad .env

# Cambiar valores:
DB_SERVER=192.168.1.50\SQLEXPRESS    # Tu servidor SQL
DB_USER=sa
DB_PASSWORD=tu_contraseña_real
```

PASO 7: Probar conexión SQL

Con el virtual environment ACTIVADO:

```python
python
>>> import pyodbc
>>> conn_str = "Driver={ODBC Driver 17 for SQL Server};Server=192.168.1.50\\SQLEXPRE
>>> conn = pyodbc.connect(conn_str)
>>> print("Conexión OK!")
>>> exit()
```

🏃 EJECUTAR LA APLICACIÓN

Modo Desarrollo (Con mocks, sin hardware)

```bash
# Asegurarse que venv esté ACTIVADO
# (venv) debe aparecer al inicio de la línea

python smart_hour_meter_v3_windows_dev.py

# Deberías ver:
# [APP] Modo Interactivo (Windows)
# [LCD] Pantalla iniciada correctamente (mock)
# [BD] Conectado a SQL Server
# [APP] Aplicación inicializada
```

Comandos en Modo Interactivo

Cuando la app esté corriendo, puedes escribir:

```
up        → Navegar arriba en menú
down      → Navegar abajo en menú
confirm   → Confirmar selección
motor     → Toggle motor ON/OFF (simular VFD)
status    → Ver estado actual
help      → Mostrar ayuda
exit      → Salir
```

Ejemplo de flujo interactivo

```
[INPUT] > confirm          # Iniciar (HOME → SELECT_EMPLOYEE)
[INPUT] > down             # Navegar a siguiente empleado
[INPUT] > confirm          # Seleccionar empleado (SELECT_TASK)
[INPUT] > down             # Navegar a siguiente tarea
[INPUT] > confirm          # Seleccionar tarea (MONITORING)
[INPUT] > motor            # Simular motor encendido
[INPUT] > confirm          # Finalizar (INPUT_COMPLETED)
[INPUT] > up up up         # Ingresar cantidad (75 piezas)
[INPUT] > confirm          # Guardar en BD
```

📝 ESTRUCTURA DE CARPETAS

```
smart-hour-meter/
├── venv/                          ← Virtual environment
│   ├── Scripts/
│   ├── Lib/
│   └── ...
│
├── smart_hour_meter_v3_rpi.py    ← Versión Raspberry Pi
├── smart_hour_meter_v3_windows_dev.py  ← Versión Windows (desarrollo)
│
├── requirements.txt               ← Dependencias
├── .env.example                   ← Configuración ejemplo
├── .env                           ← Tu configuración (NO subir a git)
│
├── consultas_sql_v3.sql          ← Consultas SQL
├── install_v3_rpi.sh             ← Script instalación RPi
├── GUIA_v3_RASPBERRY_PI.md       ← Documentación RPi
│
└── .gitignore                     ← Ignorar archivos locales
```

Archivo .gitignore (crear si no existe)

```bash
# Crear archivo .gitignore
cat > .gitignore << EOF
venv/
.env
.env.local
__pycache__/
*.pyc
*.pyo
.pytest_cache/
*.log
.vscode/
.idea/
*.swp
*.swo
EOF
```

🧪 TESTING

Verificar que todo funciona

```bash
# 1. Activar venv
.\venv\Scripts\activate

# 2. Ejecutar app
python smart_hour_meter_v3_windows_dev.py

# 3. Hacer:
# > up
# > confirm
# > down
# > confirm
# > status

# 4. Debería mostrar estado actualizado
```

Testing de conexión SQL (sin app)

```bash
```

```bash
# Python directo
python -c "import pyodbc; print('pyodbc importado correctamente')"

# O script de test
cat > test_connection.py << 'EOF'
import pyodbc

try:
    conn = pyodbc.connect(
        'Driver={ODBC Driver 17 for SQL Server};'
        'Server=192.168.1.50\\SQLEXPRESS;'
        'Database=ERP_Production;'
        'UID=sa;'
        'PWD=tu_password;'
        'TrustServerCertificate=yes'
    )
    print("✓ Conexión exitosa a SQL Server")

    cursor = conn.cursor()
    cursor.execute("SELECT COUNT(*) FROM dbo.Employee")
    count = cursor.fetchone()[0]
    print(f"✓ {count} empleados en la BD")

    conn.close()
except Exception as e:
    print(f"✗ Error: {e}")
EOF

python test_connection.py
```

🐛 TROUBLESHOOTING

Error: "No module named 'pyodbc'"

```bash
```

```
# Asegurarse que venv esté ACTIVADO
# Ver que aparezca (venv) al inicio de la línea

# Si no está activado:
.\venv\Scripts\activate

# Instalar nuevamente
pip install pyodbc
```

Error: ODBC Driver not found

```bash
# Descargar e instalar desde:
# https://www.microsoft.com/en-us/download/details.aspx?id=56567

# Verificar que esté instalado:
# Control Panel → Programs → Programs and Features
# Buscar "ODBC Driver 17 for SQL Server"

# Si no está, instalarlo
```

Error: "Cannot connect to SQL Server"

```bash
# 1. Verificar que SQL Server esté corriendo
# Services → SQL Server (SQLEXPRESS)

# 2. Verificar dirección del servidor
# SELECT @@SERVERNAME (en SQL Server Management Studio)

# 3. Verificar credenciales
# Usuario: sa
# Contraseña: (la que configuraste en instalación)

# 4. Probar conexión en SSMS primero
# New Connection → Server name: 192.168.1.50\SQLEXPRESS
```

Error: "Port already in use"

```bash
```

```
# Si algo ocupa un puerto:
# (esto es raro en esta app, pero por si acaso)

# Cambiar puerto en .env:
DB_PORT=1434  # en lugar de 1433
```

La app se cierra inmediatamente

```bash
# Ver traceback completo:
python -u smart_hour_meter_v3_windows_dev.py 2>&1 | tee app.log

# Ver contenido del log:
type app.log
```

📦 ACTUALIZAR DEPENDENCIAS

Ver versiones disponibles

```bash
# Instalar pip-tools para mejor manejo de dependencias
pip install pip-tools

# Ver actualizaciones disponibles
pip list --outdated
```

Actualizar todo

```bash
# Actualizar todas las dependencias
pip install --upgrade -r requirements.txt

# Guardar versiones actuales
pip freeze > requirements-freeze.txt
```

🔄 WORKFLOW DE DESARROLLO

Ciclo típico

```bash
# 1. Desactivar environment anterior (si cambiaste carpeta)
deactivate

# 2. Ir a carpeta del proyecto
cd C:\smart-hour-meter

# 3. Activar environment
.\venv\Scripts\activate

# 4. Editar código con tu editor favorito
# (VS Code, PyCharm, etc)

# 5. Ejecutar app
python smart_hour_meter_v3_windows_dev.py

# 6. Cuando termines, desactivar environment
deactivate
```

Usar VS Code (recomendado)

```bash
# 1. Instalar extensión "Python" en VS Code
# 2. Abrir carpeta del proyecto
# 3. Seleccionar interpreter:
#    - Ctrl+Shift+P → "Python: Select Interpreter"
#    - Elegir ".\venv\Scripts\python.exe"
# 4. Terminal → Ejecutar en Terminal → Automáticamente usa venv
```

✅ CHECKLIST FINAL

```
AMBIENTE:
☒ Python 3.9+ instalado
☒ venv creado en carpeta del proyecto
☐ venv activado (aparece (venv) en línea de comandos)
```

```
☑ requirements.txt instalado (pip install -r requirements.txt)

SQL SERVER:
☑ ODBC Driver 17 instalado
☑ SQL Server corriendo (Services)
☑ Base de datos ERP_Production existe
☑ Tablas están en su lugar

CONFIGURACIÓN:
☑ .env creado desde .env.example
☑ Credenciales SQL correctas en .env
☑ Work_Center = "TORNO 17"

TESTING:
☑ python smart_hour_meter_v3_windows_dev.py ejecuta
☐ Pantalla LCD se muestra en consola
☐ Puedo escribir comandos (up, down, confirm)
☐ Status muestra información actual

LISTO:
☐ Puedo hacer ciclo completo (SELECT → TAREA → INPUT → SAVE)
☐ Datos se guardan en BD
```

🎓 PRÓXIMOS PASOS

1. **Desarrollo en Windows**
   - Modificar código según necesidades
   - Testear lógica de grupos
   - Verificar guardado en BD
2. **Pasar a Raspberry Pi**
   - Ejecutar `install_v3_rpi.sh`
   - Cambiar pines GPIO según tu hardware
   - Instalar LCD real y botones
3. **Producción**
   - Ejecutar en Raspberry Pi
   - Configurar systemd service
   - Monitorear con journalctl

📞 SOPORTE

Si tienes problemas:

1. Ver archivo `app.log` generado
2. Ejecutar: `pip list` para verificar dependencias
3. Probar: `python test_connection.py` para SQL
4. Ver traceback completo con `-u` flag: `python -u app.py`

¡Listo! Ahora puedes desarrollar Smart Hour Meter v3 en Windows.

Próximo paso: Lee GUIA_v3_RASPBERRY_PI.md cuando estés listo para pasar a hardware real.